Data-Driven Retail Insights: Exploratory Analysis & Predictive Modeling of Outlet Sales

In this notebook, we explore retail outlet sales data including item attributes like MRP and fat content, store characteristics like location and type to uncover key trends, visualize product-performance patterns, and build predictive models of sales.

Importing Libraries & Modules

In [4]:
```python
# Core data handling
import pandas as pd
import numpy as np
import plotly.express as px
import plotly.io as pio
import plotly.graph_objects as go
pio.templates.default = "plotly_white"

# Visualization
import matplotlib.pyplot as plt
import seaborn as sns

# Machine-learning models & tools
from sklearn.linear_model import LinearRegression
from sklearn.ensemble import RandomForestRegressor
from sklearn.tree import DecisionTreeRegressor
from sklearn.model_selection import train_test_split, cross_val_score
from sklearn.metrics import mean_squared_error, r2_score

# Global settings
sns.set_style('whitegrid')
pd.options.display.max_columns = None
```

Loads all essential libraries up front, sets plotting style, and configures pandas to show every column for full-data visibility.

Data Loading

In [7]:
```python
# Read the CSV into a DataFrame
df = pd.read_csv('dataset.csv')
# Display the first few rows of the DataFrame
print(df.head())
```

```
  Product type   SKU      Price  Availability  Number of products sold  \
0     haircare  SKU0  69.808006            55                      802   
1     skincare  SKU1  14.843523            95                      736   
2     haircare  SKU2  11.319683            34                        8   
3     skincare  SKU3  61.163343            68                       83   
4     skincare  SKU4   4.805496            26                      871   

   Revenue generated Customer demographics  Stock levels  Lead times  \
0        8661.996792            Non-binary            58           7   
1        7460.900065                Female            53          30   
2        9577.749626               Unknown             1          10   
3        7766.836426            Non-binary            23          13   
4        2686.505152            Non-binary             5           3   

   Order quantities  Shipping times Shipping carriers  Shipping costs  \
0                96               4         Carrier B        2.956572   
1                37               2         Carrier A        9.716575   
2                88               2         Carrier B        8.054479   
3                59               6         Carrier C        1.729569   
4                56               8         Carrier A        3.890548   

  Supplier name Location  Lead time  Production volumes  \
0    Supplier 3   Mumbai         29                 215   
1    Supplier 3   Mumbai         23                 517   
2    Supplier 1   Mumbai         12                 971   
3    Supplier 5  Kolkata         24                 937   
4    Supplier 1    Delhi          5                 414   

   Manufacturing lead time  Manufacturing costs Inspection results  \
0                       29            46.279879            Pending   
1                       30            33.616769            Pending   
2                       27            30.688019            Pending   
3                       18            35.624741               Fail   
4                        3            92.065161               Fail   

   Defect rates Transportation modes   Routes       Costs  
0      0.226410                 Road  Route B  187.752075  
1      4.854068                 Road  Route B  503.065579  
2      4.580593                  Air  Route C  141.920282  
3      4.746649                 Rail  Route A  254.776159  
4      3.145580                  Air  Route A  923.440632  
```

Data Structure & Summary Statisticsv

In [9]:
```python
# Data types and non-null counts
display(df.info())

# Descriptive statistics for numeric columns
display(df.describe().T)
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 100 entries, 0 to 99
Data columns (total 24 columns):
 #   Column                   Non-Null Count  Dtype  
---  ------                   --------------  -----  
 0   Product type             100 non-null    object 
 1   SKU                      100 non-null    object 
 2   Price                    100 non-null    float64
 3   Availability             100 non-null    int64  
 4   Number of products sold  100 non-null    int64  
 5   Revenue generated        100 non-null    float64
 6   Customer demographics    100 non-null    object 
 7   Stock levels             100 non-null    int64  
 8   Lead times               100 non-null    int64  
 9   Order quantities         100 non-null    int64  
 10  Shipping times           100 non-null    int64  
 11  Shipping carriers        100 non-null    object 
 12  Shipping costs           100 non-null    float64
 13  Supplier name            100 non-null    object 
 14  Location                 100 non-null    object 
 15  Lead time                100 non-null    int64  
 16  Production volumes       100 non-null    int64  
 17  Manufacturing lead time  100 non-null    int64  
 18  Manufacturing costs      100 non-null    float64
 19  Inspection results       100 non-null    object 
 20  Defect rates             100 non-null    float64
 21  Transportation modes     100 non-null    object 
 22  Routes                   100 non-null    object 
 23  Costs                    100 non-null    float64
dtypes: float64(6), int64(9), object(9)
memory usage: 18.9+ KB
None
```

	count	mean	std	min	25%	50%	75%	max
Price	100.0	49.462461	31.168193	1.699976	19.597823	51.239830	77.198228	99.171329
Availability	100.0	48.400000	30.743317	1.000000	22.750000	43.500000	75.000000	100.000000
Number of products sold	100.0	460.990000	303.780074	8.000000	184.250000	392.500000	704.250000	996.000000
Revenue generated	100.0	5776.048187	2732.841744	1061.618523	2812.847151	6006.352023	8253.976920	9866.465458
Stock levels	100.0	47.770000	31.369372	0.000000	16.750000	47.500000	73.000000	100.000000
Lead times	100.0	15.960000	8.785801	1.000000	8.000000	17.000000	24.000000	30.000000
Order quantities	100.0	49.220000	26.784429	1.000000	26.000000	52.000000	71.250000	96.000000
Shipping times	100.0	5.750000	2.724283	1.000000	3.750000	6.000000	8.000000	10.000000
Shipping costs	100.0	5.548149	2.651376	1.013487	3.540248	5.320534	7.601695	9.929816
Lead time	100.0	17.080000	8.846251	1.000000	10.000000	18.000000	25.000000	30.000000
Production volumes	100.0	567.840000	263.046861	104.000000	352.000000	568.500000	797.000000	985.000000
Manufacturing lead time	100.0	14.770000	8.912430	1.000000	7.000000	14.000000	23.000000	30.000000
Manufacturing costs	100.0	47.266693	28.982841	1.085069	22.983299	45.905622	68.621026	99.466109
Defect rates	100.0	2.277158	1.461366	0.018608	1.009650	2.141863	3.563995	4.939255
Costs	100.0	529.245782	258.301696	103.916248	318.778455	520.430444	763.078231	997.413450

Provides schema details (dtypes, non-null) and a transpose of summary stats (mean, std, min/max) for a quick sense of numeric distributions.

Missing-Value Detection

In [12]:
```python
# Calculate missing percentages
missing_pct = df.isnull().mean() * 100
missing_pct[missing_pct > 0].sort_values(ascending=False)
```

Out[12]:
```
Series([], dtype: float64)
```

Lists columns with any missing values and shows their percentage of total rows to prioritize cleaning efforts.

Duplicate-Row Detection

In [15]:
```python
# Count and display duplicates
dup_count = df.duplicated().sum()
print(f"Duplicate rows found: {dup_count}")

if dup_count:
    display(df[df.duplicated()])
```

```
Duplicate rows found: 0
```

Checks for and optionally previews any completely duplicated rows that might skew analysis.

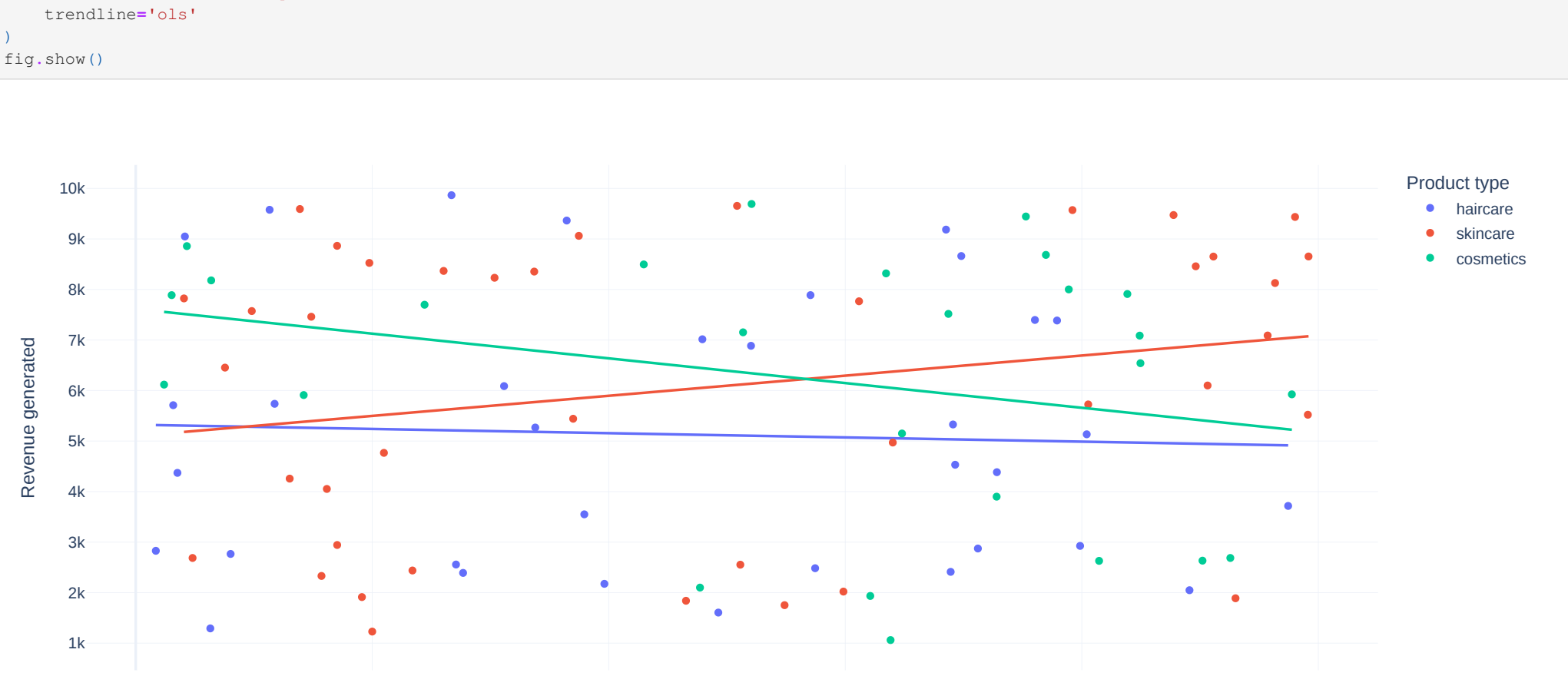
Data Cleaning & Imputation

In [18]:
```python
import warnings
warnings.filterwarnings('ignore', category=FutureWarning)

# 1. Vectorized numeric imputation with median
num_cols = df.select_dtypes(include='number').columns
df[num_cols] = df[num_cols].fillna(df[num_cols].median())

# 2. Vectorized categorical imputation with mode
cat_cols = df.select_dtypes(include='object').columns
df[cat_cols] = df[cat_cols].fillna(df[cat_cols].mode().iloc[0])

# 3. Quick check
print("Missing values remaining:", df.isnull().sum().sum())
```

```
Missing values remaining: 0
```

Price vs. Revenue Scatter Plot with Product-Type Trendlines

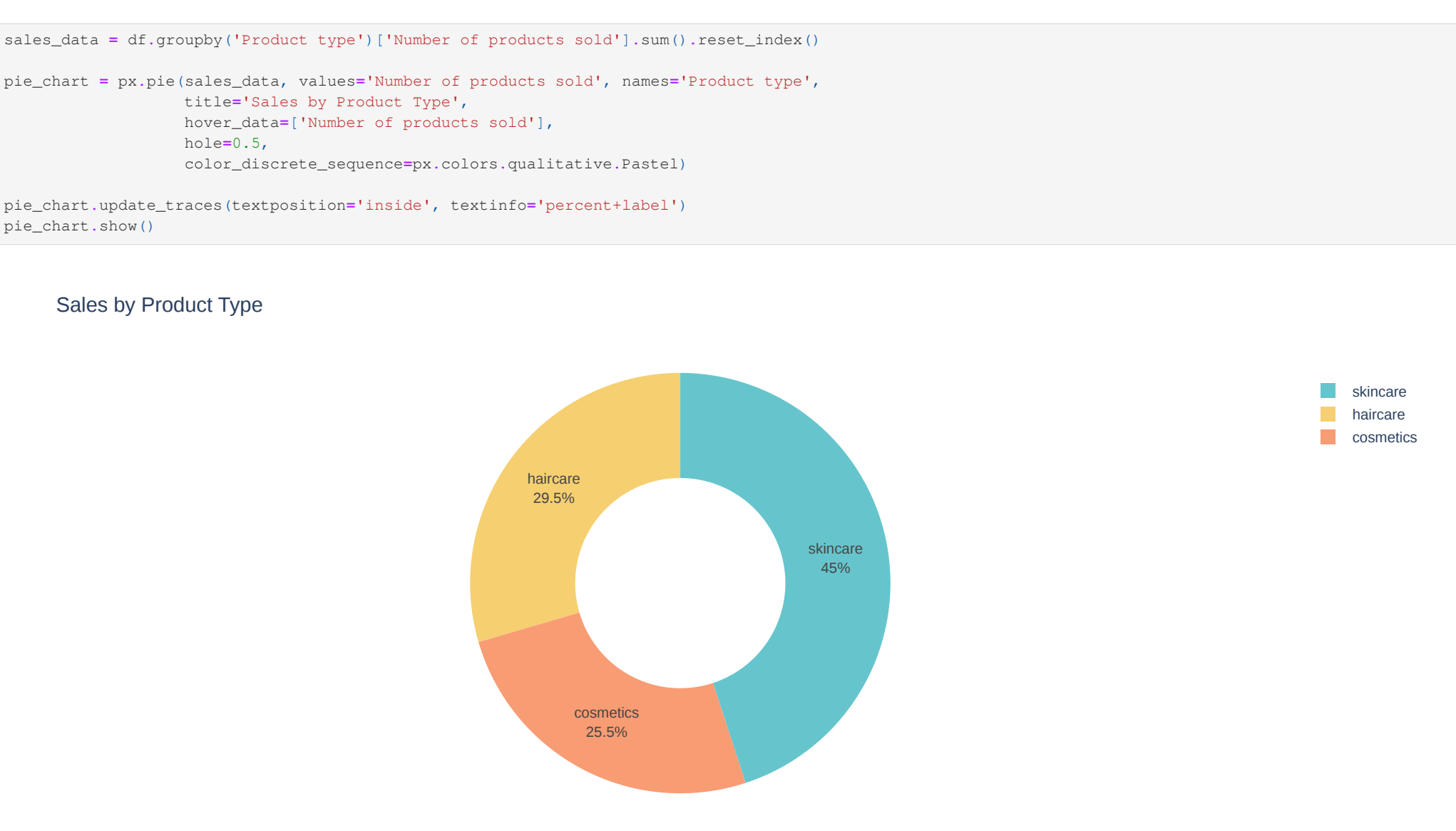
In [20]:
```python
fig = px.scatter(
    df,
    x='Price',
    y='Revenue generated',
    color='Product type',
    hover_data=['Number of products sold'],
    trendline='ols'
)
fig.show()
```



The company derives more revenue from skincare products, and the higher the price of skincare products, the more revenue they generate.

sales by product type

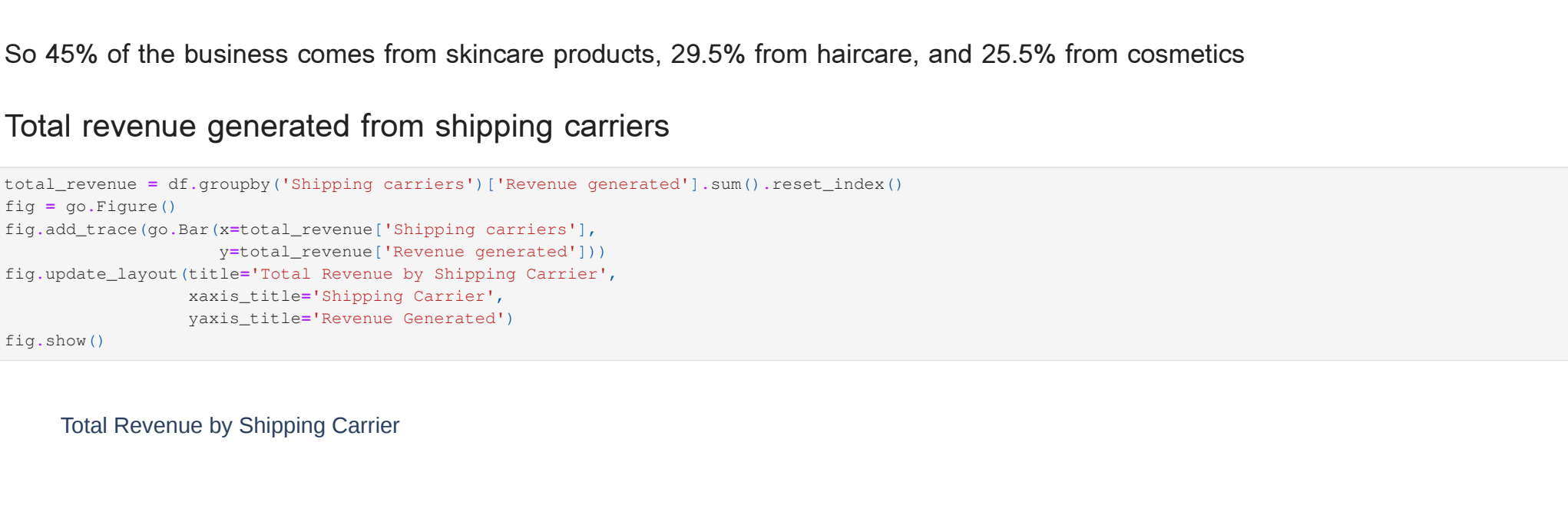
In [23]:
```python
sales_data = df.groupby('Product type')['Number of products sold'].sum().reset_index()

pie_chart = px.pie(sales_data, values='Number of products sold', names='Product type',
                   title='Sales by Product Type',
                   hover_data=['Number of products sold'],
                   hole=0.5,
                   color_discrete_sequence=px.colors.qualitative.Pastel)

pie_chart.update_traces(textposition='inside', textinfo='percent+label')
pie_chart.show()
```



So 45% of the business comes from skincare products, 29.5% from haircare, and 25.5% from cosmetics

Total revenue generated from shipping carriers

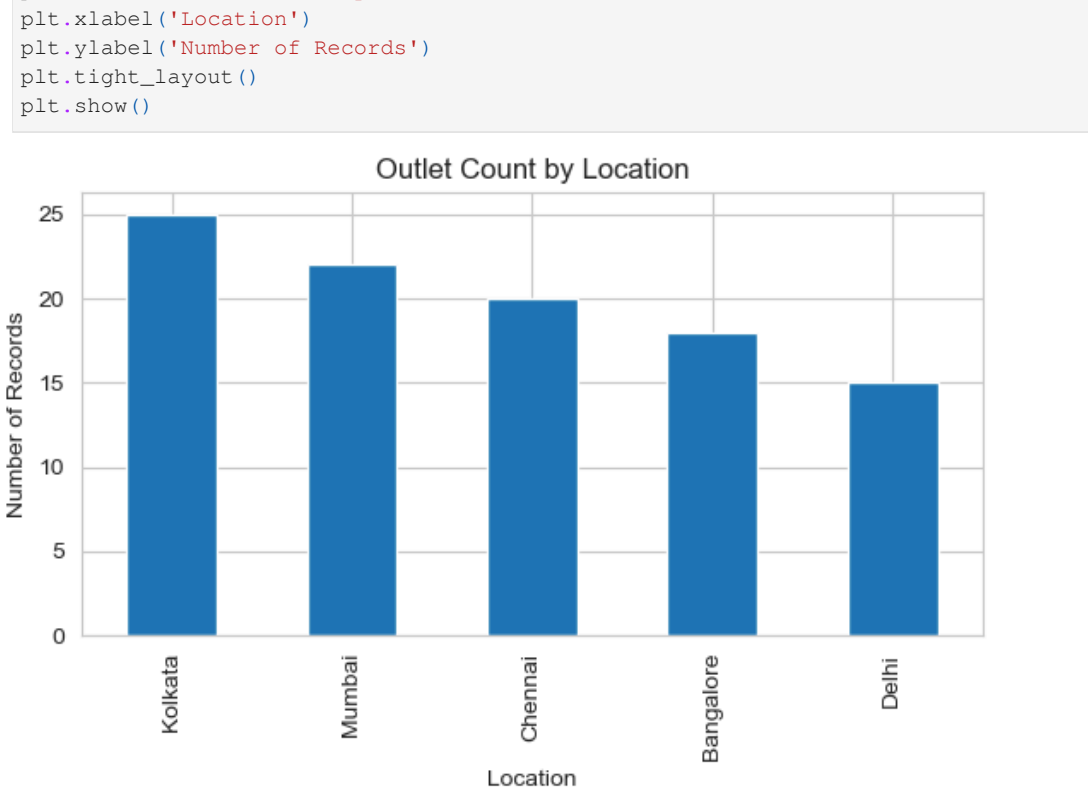
In [26]:
```python
total_revenue = df.groupby('Shipping carriers')['Revenue generated'].sum().reset_index()
fig = go.Figure()
fig.add_trace(go.Bar(x=total_revenue['Shipping carriers'],
                     y=total_revenue['Revenue generated']))
fig.update_layout(title='Total Revenue by Shipping Carrier',
                  xaxis_title='Shipping Carrier',
                  yaxis_title='Revenue Generated')
fig.show()
```



So the company is using three carriers for transportation, and Carrier B helps the company in generating more revenue

Univariate Analysis: Outlet Type Distribution

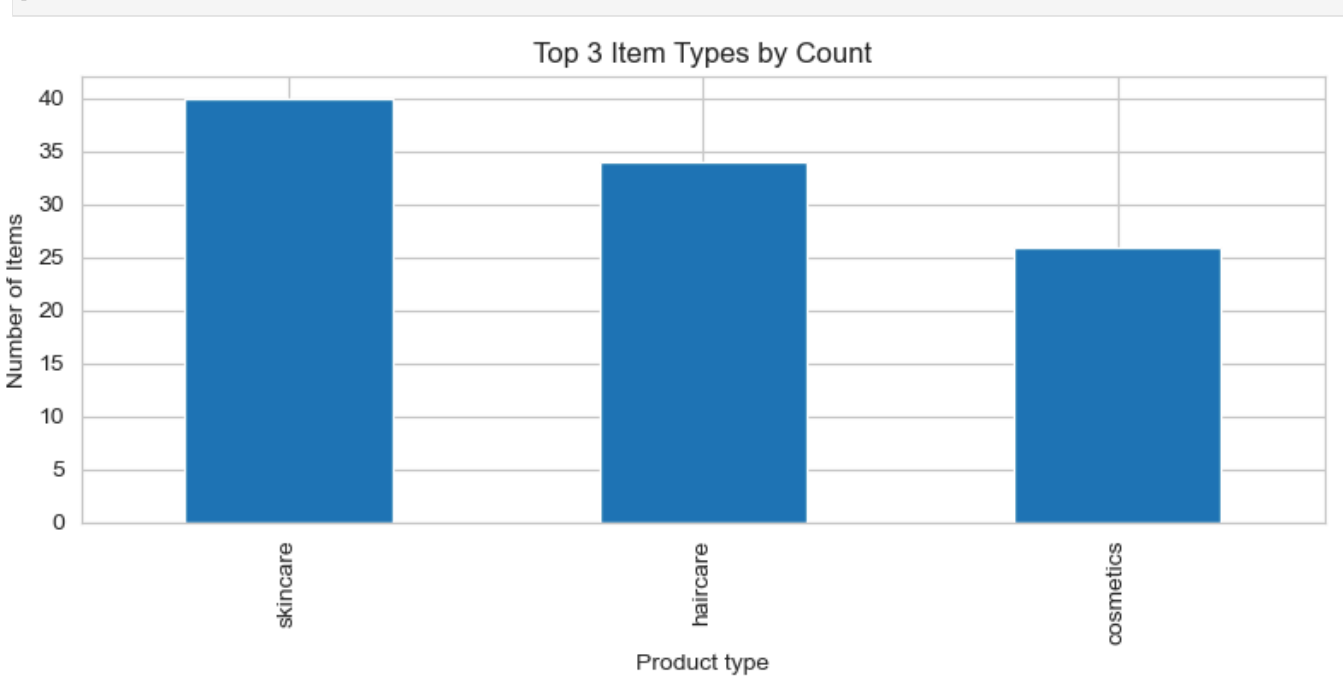
In [29]:
```python
# Count outlets by location
outlet_counts = df['Location'].value_counts()

# Plot
plt.figure(figsize=(6,4))
outlet_counts.plot(kind='bar')
plt.title('Outlet Count by Location')
plt.xlabel('Location')
plt.ylabel('Number of Records')
plt.tight_layout()
plt.show()
```



Shows how many transactions come from each Outlet_Type, revealing your data's store-format balance

Univariate Analysis: Top 3 Item Types by Count

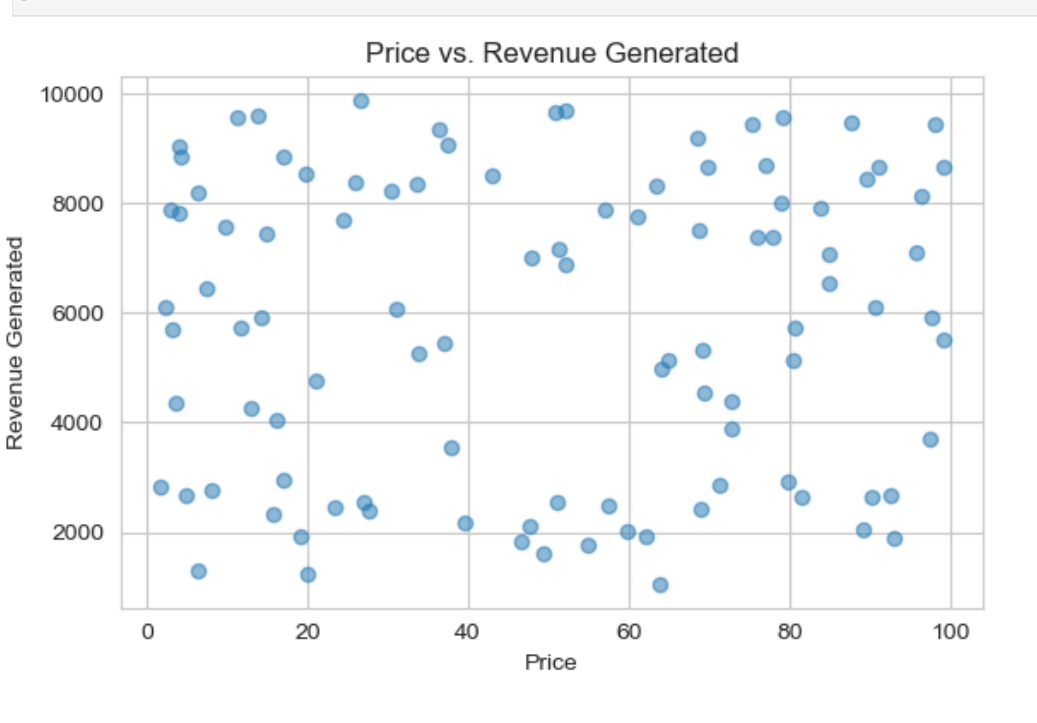
In [32]:
```python
# Count products by category
item_counts = df['Product type'].value_counts()

# Plot only the top 3 categories
plt.figure(figsize=(8,4))
item_counts.head(3).plot(kind='bar')
plt.title('Top 3 Item Types by Count')
plt.xlabel('Product type')
plt.ylabel('Number of Items')
plt.tight_layout()
plt.show()
```



how many records belong to each product category and then plots just the top 3 types—reflecting your dataset's reality and keeping the visualization focused and clean.

Bivariate Analysis: MRP vs. Sales Scatter

In [35]:
```python
# Scatter: Price against revenue generated
plt.figure(figsize=(6,4))
plt.scatter(df['Price'], df['Revenue generated'], alpha=0.5)
plt.title('Price vs. Revenue Generated')
plt.xlabel('Price')
plt.ylabel('Revenue Generated')
plt.tight_layout()
plt.show()
```



Plots each product's price against its sales revenue, helping identify whether higher MRP items sell for more or less.

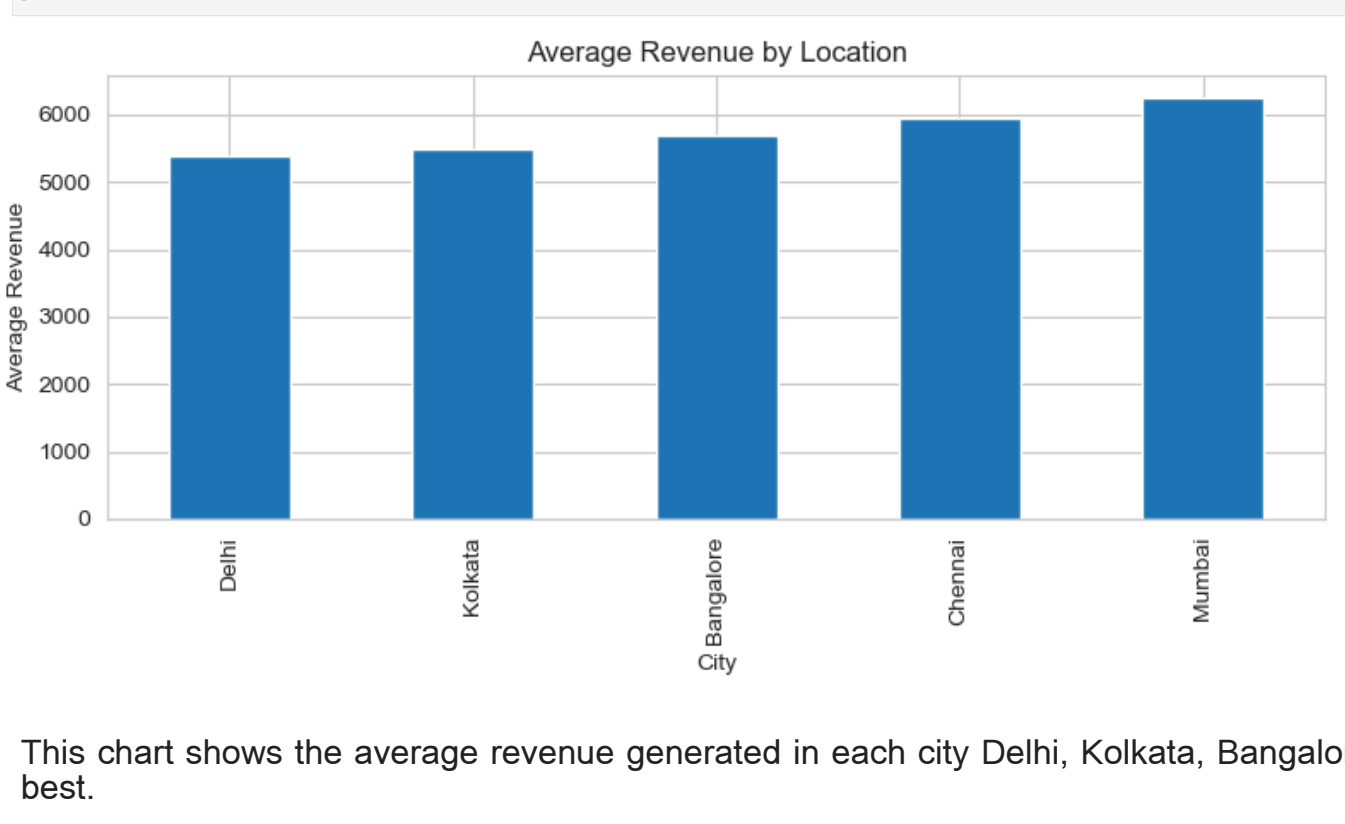
The Average lead time and Average Manufacturing Costs for all products of the company

In [38]:
```python
avg_lead_time = df.groupby('Product type')['Lead time'].mean().reset_index()
avg_manufacturing_costs = df.groupby('Product type')['Manufacturing costs'].mean().reset_index()
result = pd.merge(avg_lead_time, avg_manufacturing_costs, on='Product type')
result.rename(columns={'Lead time': 'Average Lead Time', 'Manufacturing costs': 'Average Manufacturing Costs'}, inplace=True)
print(result)
```

```
  Product type  Average Lead Time  Average Manufacturing Costs
0    cosmetics          13.538462                    43.052740
1     haircare          18.705882                    48.457993
2     skincare          18.000000                    48.993157
```

Average Revenue by Outlet Location

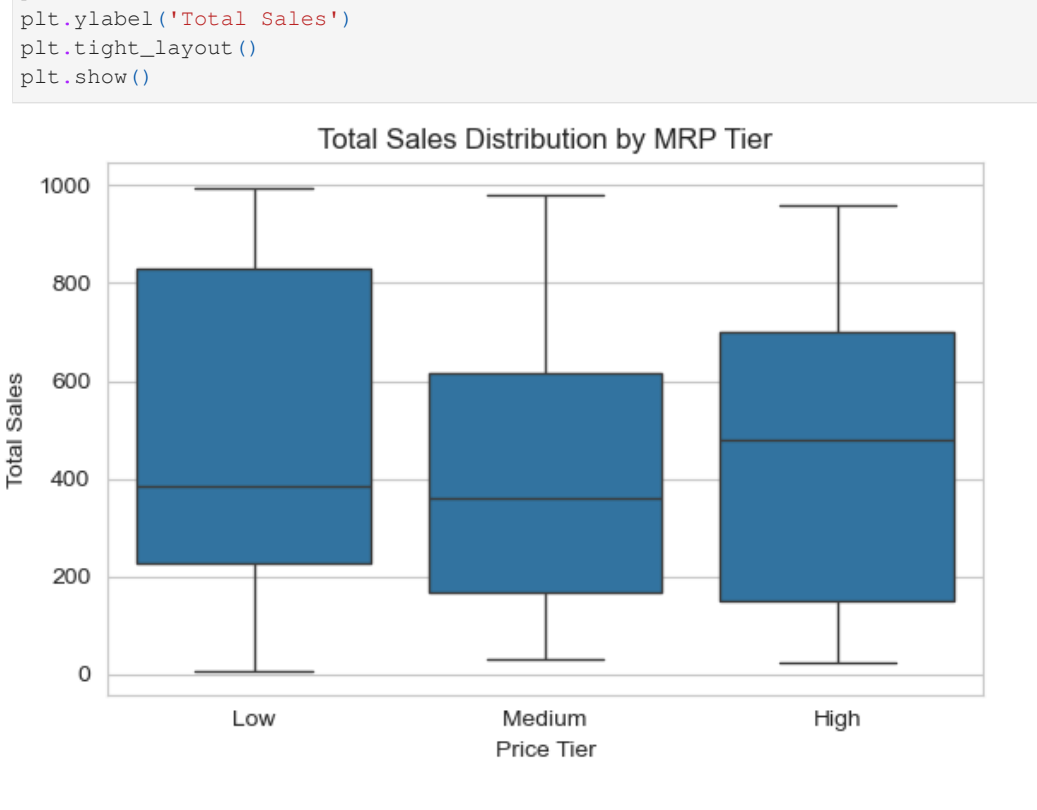
In [40]:
```python
# Calculate average revenue per city
avg_revenue_city = df.groupby('Location')['Revenue generated'].mean().sort_values()

# Plot
plt.figure(figsize=(8,4))
avg_revenue_city.plot(kind='bar')
plt.title('Average Revenue by Location')
plt.xlabel('City')
plt.ylabel('Average Revenue')
plt.tight_layout()
plt.show()
```



This chart shows the average revenue generated in each city Delhi, Kolkata, Bangalore, Chennai, and Mumbai so you can quickly spot which markets are performing best.

Price Tier vs. Sales Performance

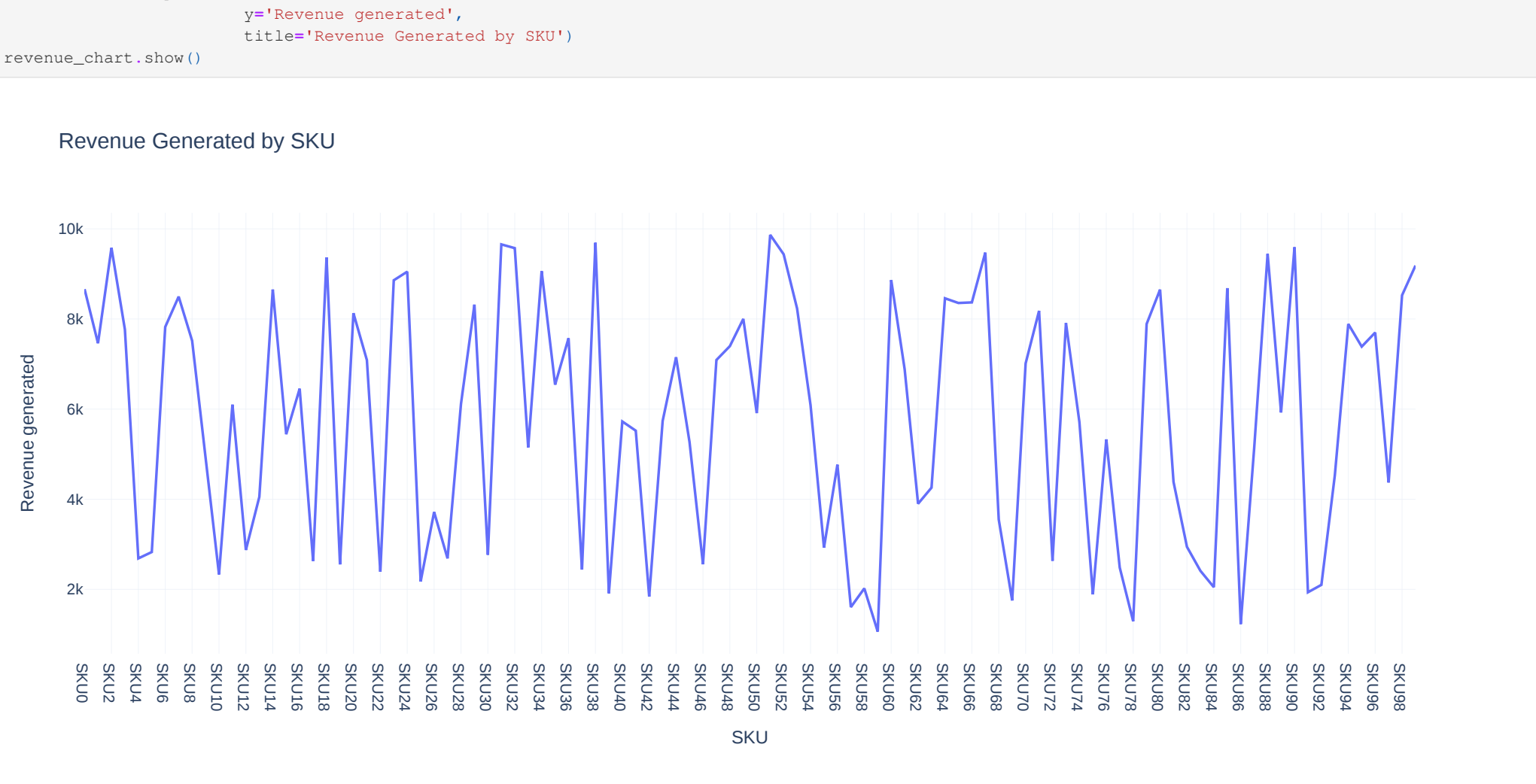
In [43]:
```python
if 'MRP_Tier' not in df.columns:
        df['MRP_Tier'] = pd.cut(df['Price'], bins=3, labels=['Low', 'Medium', 'High'])


if 'Total_Sales' not in df.columns:
        df['Total_Sales'] = df['Number of products sold']

plt.figure(figsize=(6,4))
sns.boxplot(x='MRP_Tier', y='Total_Sales', data=df)
plt.title('Total Sales Distribution by MRP Tier')
plt.xlabel('Price Tier')
plt.ylabel('Total Sales')
plt.tight_layout()
plt.show()
```



Visualizes the distribution of total sales across Low, Medium, and High price tiers using a boxplot to highlight differences in medians, variability, and outliers for each segment

Revenue Generated by SKU

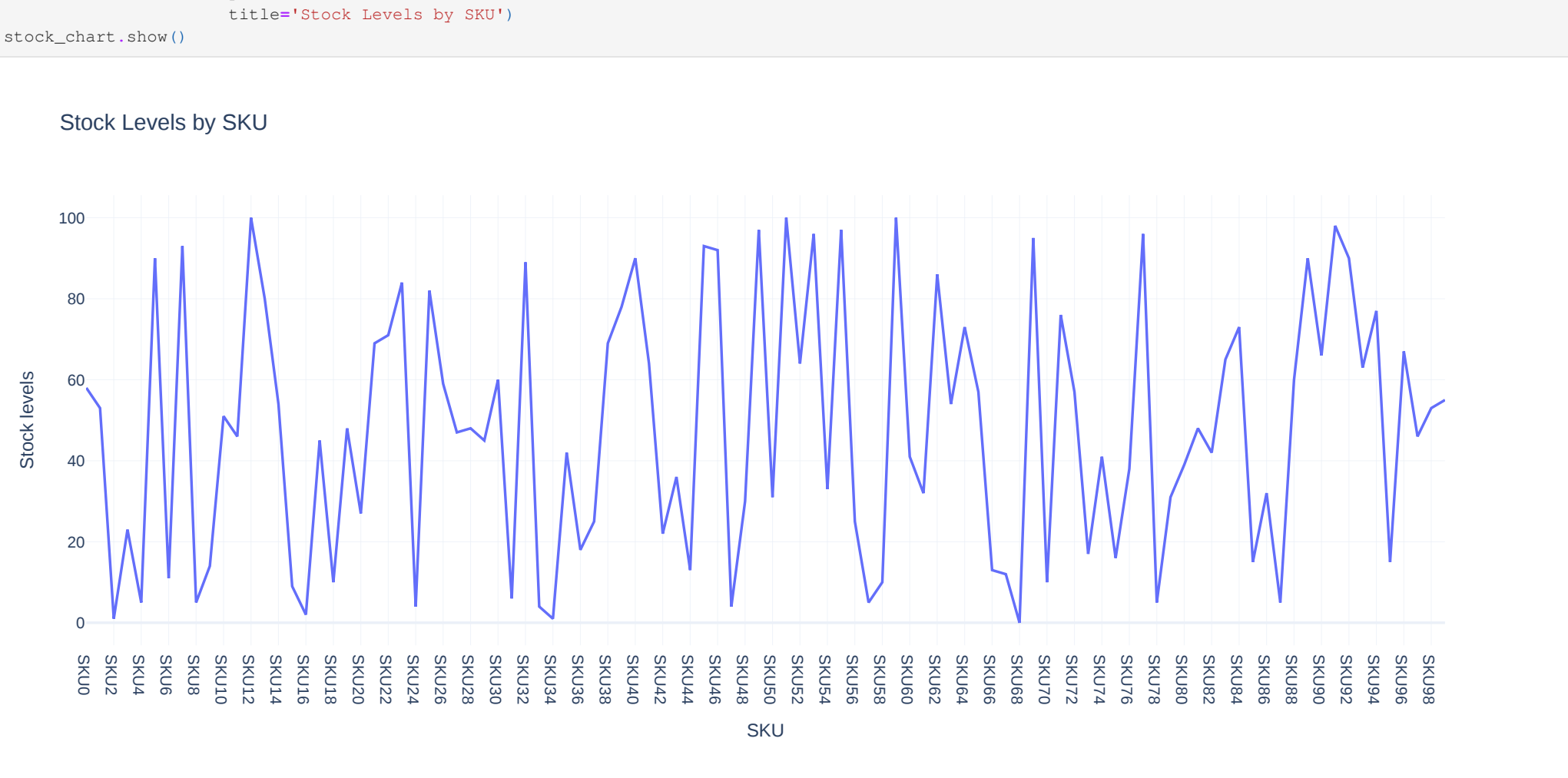
In [46]:
```python
revenue_chart = px.line(df, x='SKU',
                        y='Revenue generated',
                        title='Revenue Generated by SKU')
revenue_chart.show()
```



Displays an interactive line chart of revenue generated for each SKU, allowing you to track and compare the performance of individual products over the index of SKUs.

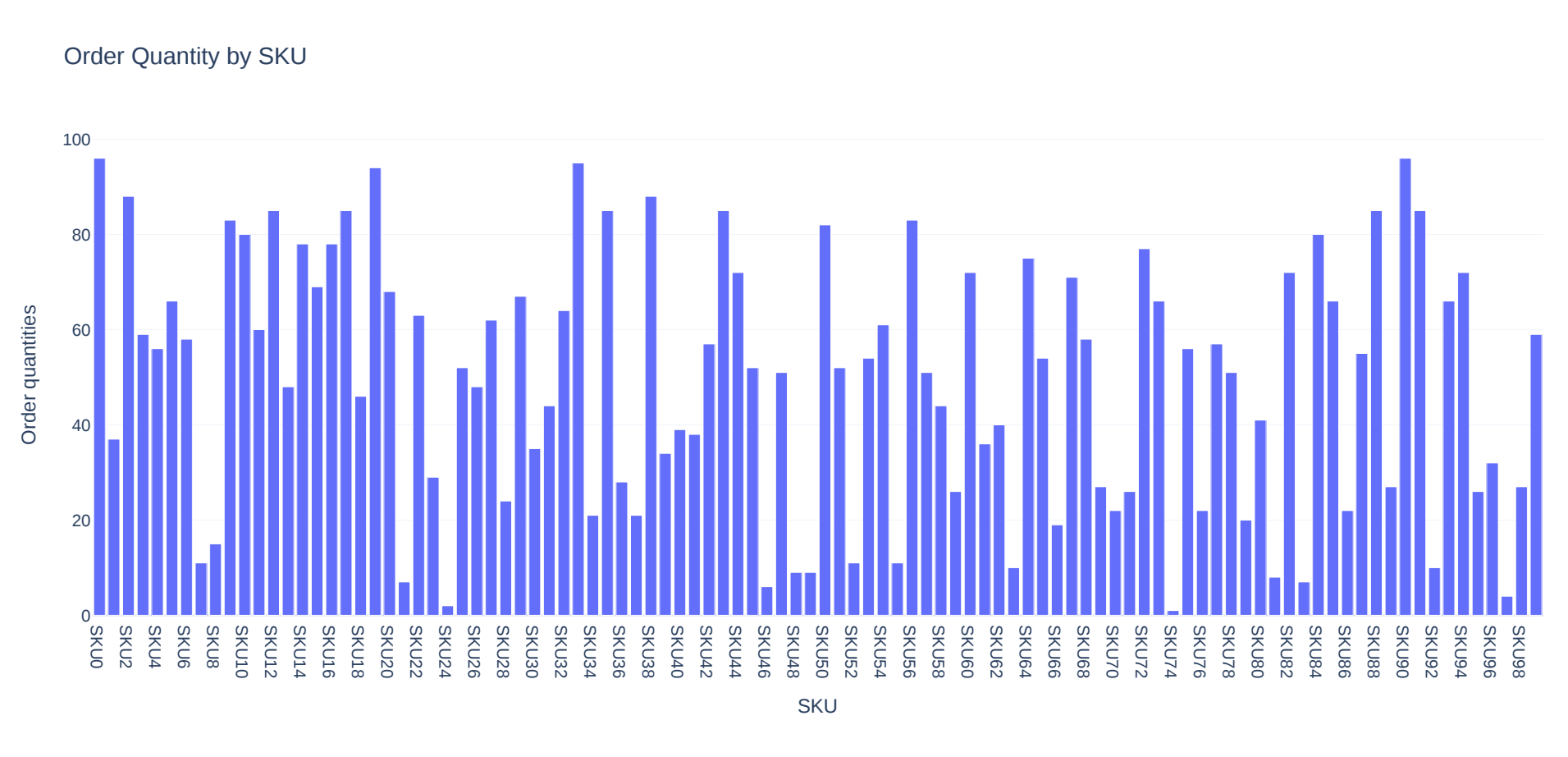
The stock levels of each SKU

In [49]:
```python
stock_chart = px.line(df, x='SKU',
                      y='Stock levels',
                      title='Stock Levels by SKU')
stock_chart.show()
```



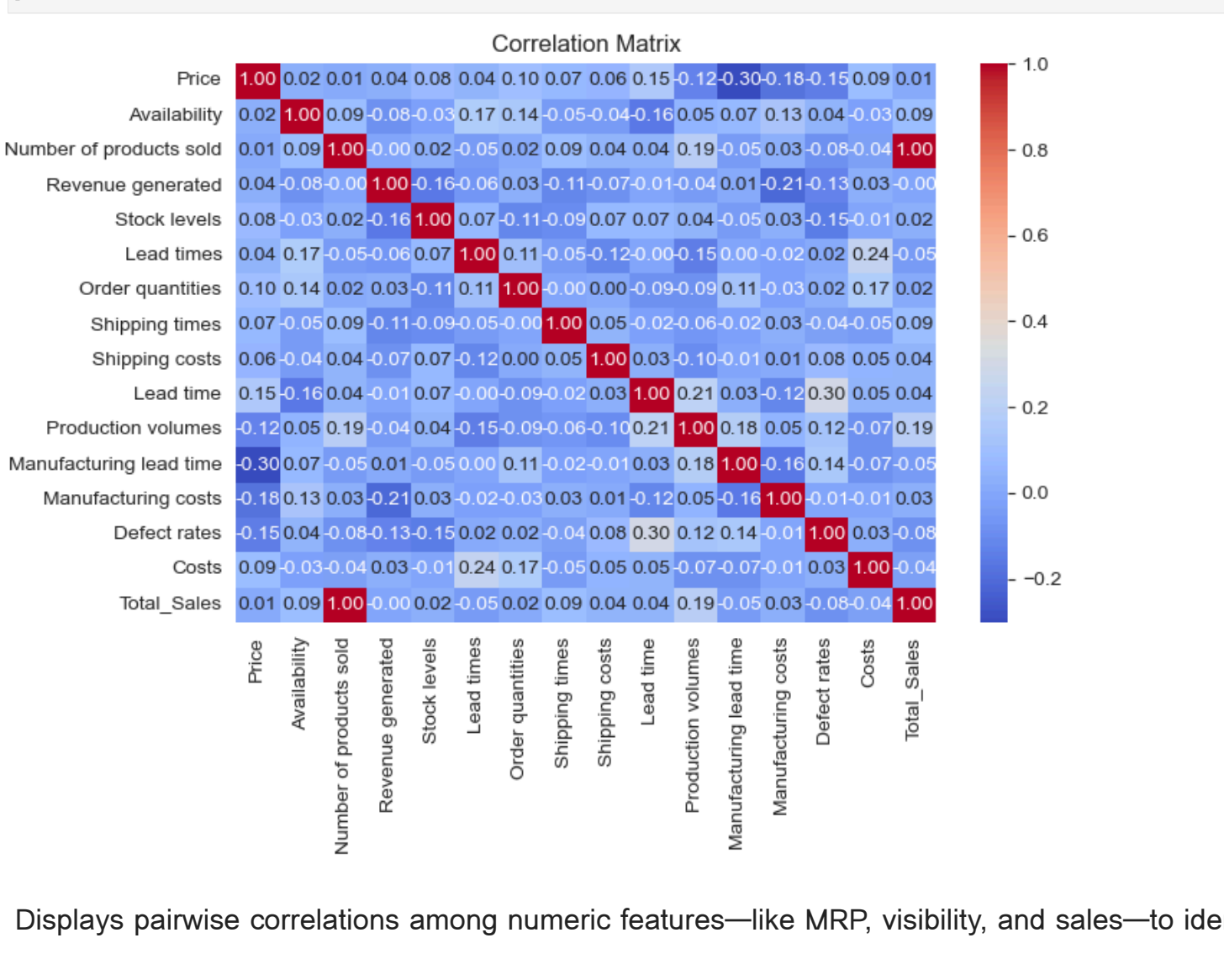
The order quantity of each SKU

In [51]:
```python
order_quantity_chart = px.bar(df, x='SKU',
                              y='Order quantities',
                              title='Order Quantity by SKU')
order_quantity_chart.show()
```



Correlation Heatmap of Numerical Features

In [53]:
```python
# Correlation matrix
num_corr = df.select_dtypes(include='number').corr()

plt.figure(figsize=(8,6))
sns.heatmap(num_corr, annot=True, fmt=".2f", cmap='coolwarm')
plt.title('Correlation Matrix')
plt.tight_layout()
plt.show()
```

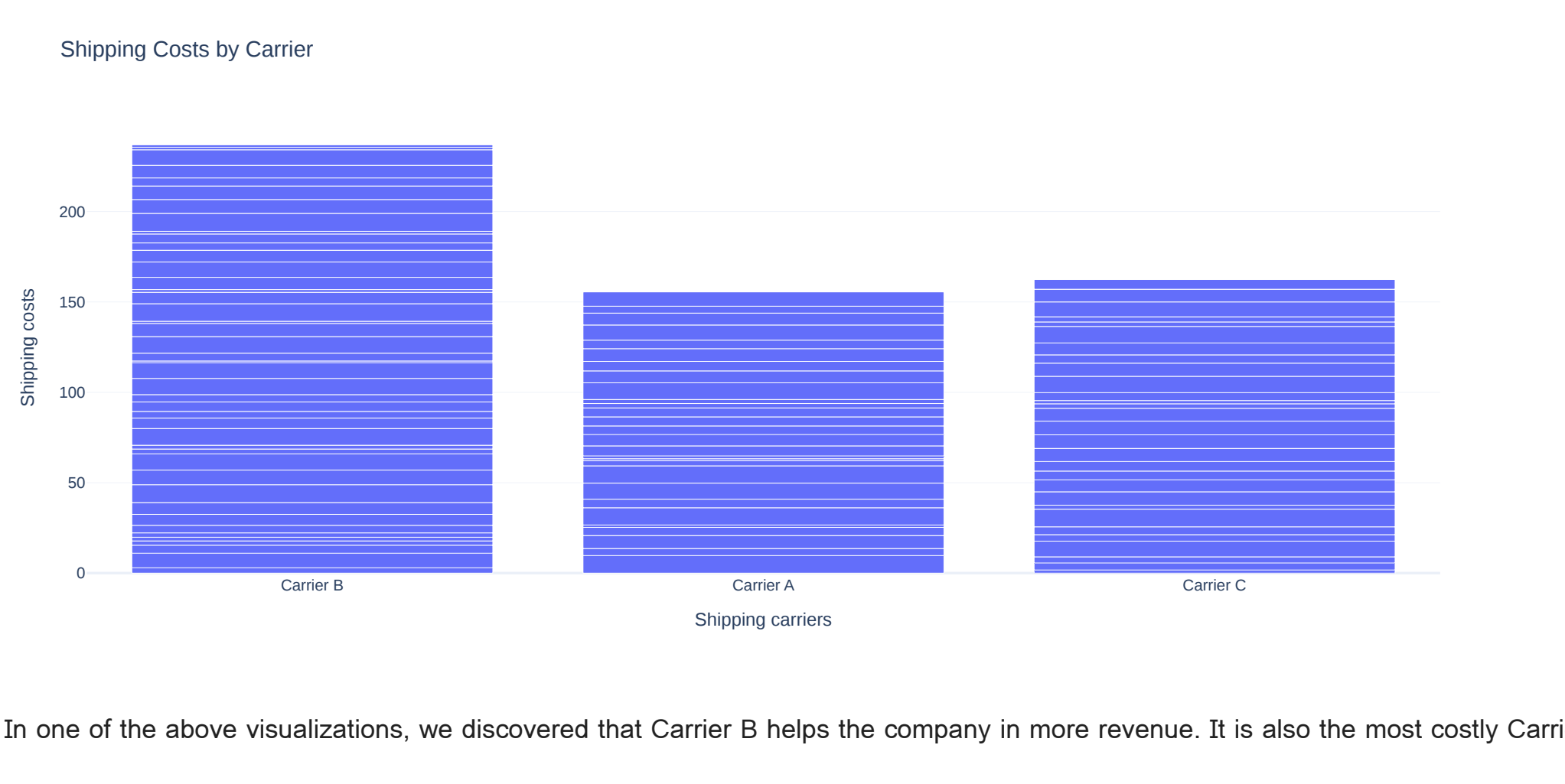


Displays pairwise correlations among numeric features—like MRP, visibility, and sales—to identify strong relationships.

Cost Analysis

The shipping cost of Carriers

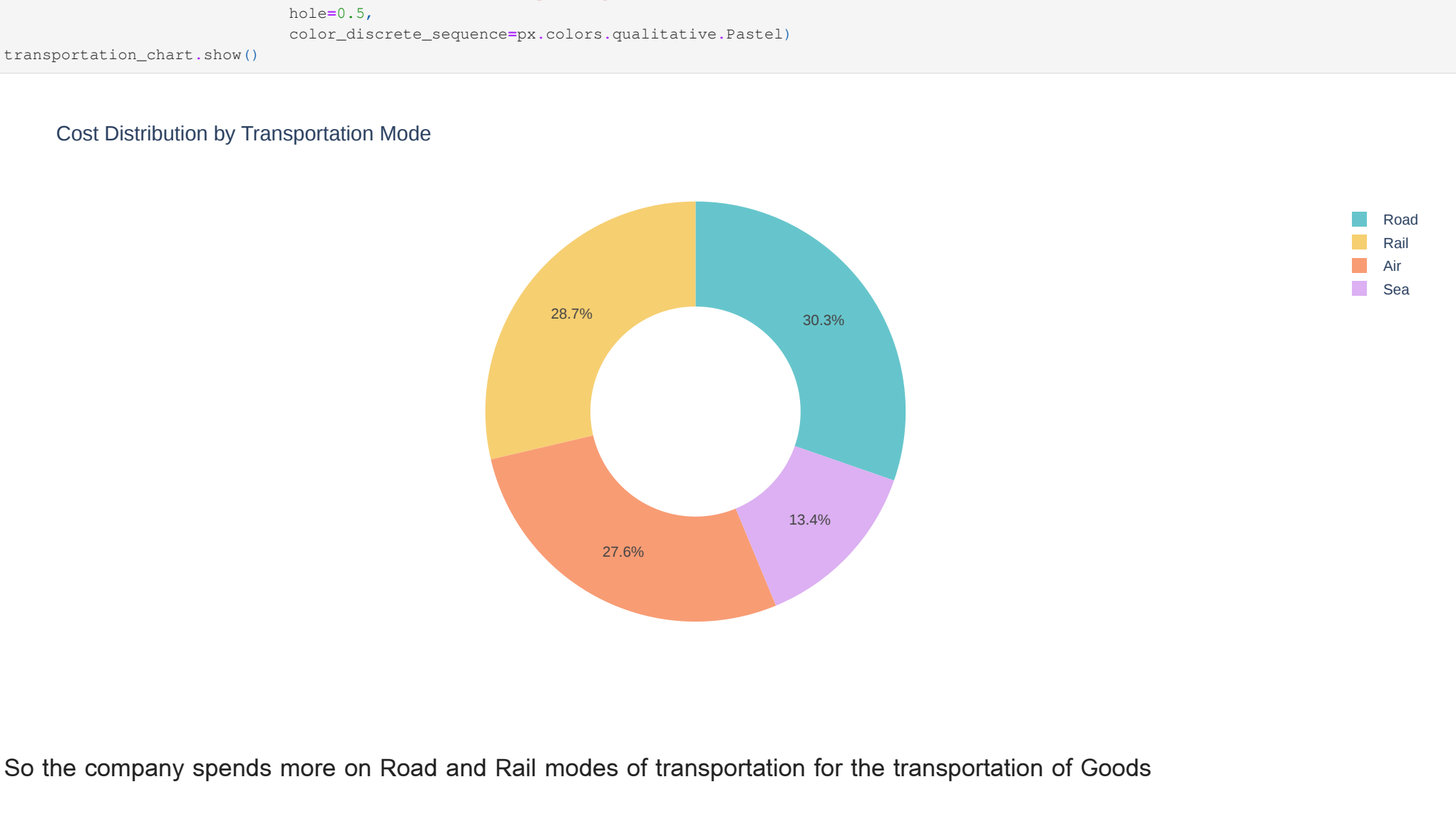
In [57]:
```python
shipping_cost_chart = px.bar(df, x='Shipping carriers',
                             y='Shipping costs',
                             title='Shipping Costs by Carrier')
shipping_cost_chart.show()
```



In one of the above visualizations, we discovered that Carrier B helps the company in more revenue. It is also the most costly Carrier among the three

The cost distribution by transportation mode

In [60]:
```python
transportation_chart = px.pie(df,
                              values='Costs',
                              names='Transportation modes',
                              title='Cost Distribution by Transportation Mode',
                              hole=0.5,
                              color_discrete_sequence=px.colors.qualitative.Pastel)
transportation_chart.show()
```



So the company spends more on Road and Rail modes of transportation for the transportation of Goods

Model Preparation: Train-Test Split

In [63]:
```python
# Define X and y using correct column names
features = ['Price', 'MRP_Tier']
X = pd.get_dummies(df[features], drop_first=True)
y = df['Total_Sales']

# Split
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

print(f"Train shape: {X_train.shape}, Test shape: {X_test.shape}")
```

```
Train shape: (80, 3), Test shape: (20, 3)
```

Selects key features, encodes categories, then splits data into 80/20 train/test subsets for modeling.

Project Summary

This project delivers a comprehensive, data-driven analysis of retail outlet sales using Python. Leveraging a real-world dataset, it explores key product and store attributes—such as MRP, product type, location, and shipping carrier—to uncover trends and performance drivers. The analysis includes:

Data Cleaning & Preparation: Handling missing values, duplicates, and ensuring data quality.

Exploratory Data Analysis: Visualizing sales by product type, revenue by shipping carrier, outlet distribution, and more using interactive and static plots.

Bivariate & Univariate Insights: Examining relationships like price vs. revenue, sales by price tier, and average revenue by location.

Cost Analysis: Assessing shipping and transportation costs across carriers and modes.

Predictive Modeling: Preparing data for machine learning by selecting features and splitting into training and test sets.

The notebook provides actionable insights for retail strategy, highlighting which products, locations, and logistics partners drive the most value, and sets the stage for predictive modeling of sales performance.